# Enhancing Federated Learning Security: Combating Clients' Data Poisoning with Classifier Ensembles

Arunava Roy, Member IEEE
*Dept. of Computer Science*
*The University of Memphis*
Memphis-38152, TN, USA
aroy@memphis.edu

Dipankar Dasgupta, Fellow IEEE
*Dept. of Computer Science*
*The University of Memphis*
Memphis-38152, TN, USA
dasgupta@memphis.edu

***Abstract*—Federated Learning (FL) enables decentralized model training while preserving data privacy, but it is vulnerable to data poisoning attacks where adversaries manipulate local data to compromise the global model. To address this, we propose a robust defense using an Ensemble of Classifiers (`EoCl`), a lightweight global input defense mechanism against clients' data poisoning. The `EoCl` is trained centrally at the CAS and distributed to clients, filtering corrupted data before it affects local models. This approach reduces the computational load on clients while enhancing their private data security and model accuracy. We recommend regularly updating the global `EoCl` by both refining the selection of classifiers and training it on a broader range of datasets. Additionally, it is advisable to periodically refresh the local `EoCl`s on active clients after a predetermined number of global iterations. The timing of these updates can be based on the quality of the clients' parameter updates and loss values, though the final decision rests with the FL service provider. This adaptive strategy helps to maintain both the accuracy and security of the FL system against clients' data poisoning attacks. We conducted extensive experiments using a variety of public datasets.**

***Index Terms*—Federated Learning, Distributed Learning, Data Poisoning, Security, Ensemble of Classifiers.**

## I. Introduction

Federated learning (FL) has emerged as a transformative approach to collaborative ML, facilitating model training across numerous decentralized clients, each maintaining its local private training data, all while safeguarding data privacy [12]. The clients first register with the Centralized Aggregate Server (CAS) to participate in FL. The CAS then selects a random subset of clients for each global iteration, which we call the set of active clients. These active clients receive the global model as a local replica (we call them local learning models) on their devices and conduct a pre-defined number of local iterations before sending their parameter updates back to the CAS. The clients' parameter updates are then aggregated at the CAS to enhance the global learning process without compromising the privacy of the clients' data. This decentralized paradigm offers several advantages, including privacy preservation and reduced communication costs [12]. However, FL also faces significant challenges and security flaws that must be addressed for its widespread adoption and effectiveness in real-world applications.

One of the primary security concerns in FL is the risk of client private data poisoning [2], [8]. Adversaries (including outside attackers and apparently trusted looking malicious client insiders) may manipulate their local training data by adding or modifying samples with the intention of influencing the model's learning process during the aggregation of parameter updates at the central server [23]. This can lead to biased model updates and reduced overall model accuracy, potentially undermining the effectiveness of the FL system. To mitigate the impact of local client data poisoning attacks, researchers have proposed various defense mechanisms. One approach involves robust aggregation methods, such as trimmed mean and median, which can identify and filter out malicious updates during the aggregation process, thereby reducing the influence of poisoned data on the global model [3]. Although such trimmed methods might remove useful data points and relies heavily on how much data is trimmed, which can be tricky to set correctly. It also assumes data is evenly distributed, which is not always true, and can be slow for large datasets. The median method is not very effective against subtle attacks, needs a lot of data to work well, and ignores the overall data pattern. Both trimmed methods might struggle with advanced attacks, need regular updates, and should be used alongside other protective measures for better security. Advanced secure communication protocols and cryptographic techniques can protect FL systems from unauthorized access and tampering with clients' sensitive data and local model parameters [4]. However, while they secure data during transmission, they do not inspect its content, allowing malicious data to be securely transmitted and integrated undetected. Additionally, these protocols add overhead, complicating real-time data analysis and lacking mechanisms to identify poisoned data, thereby necessitating additional defensive measures.

In addition to other threats, such as poisoning clients' local learning models to disrupt the training of Federated Learning (FL) [2], there are challenges due to the diverse nature of client devices. These devices vary in memory, computing power, software, and network connectivity, and their data may be incomplete or imbalanced with different types like text, images, and videos. These issues can lead to problems with synchronization, energy use, scalability, accuracy, and performance [13], though they are not the focus of this work.

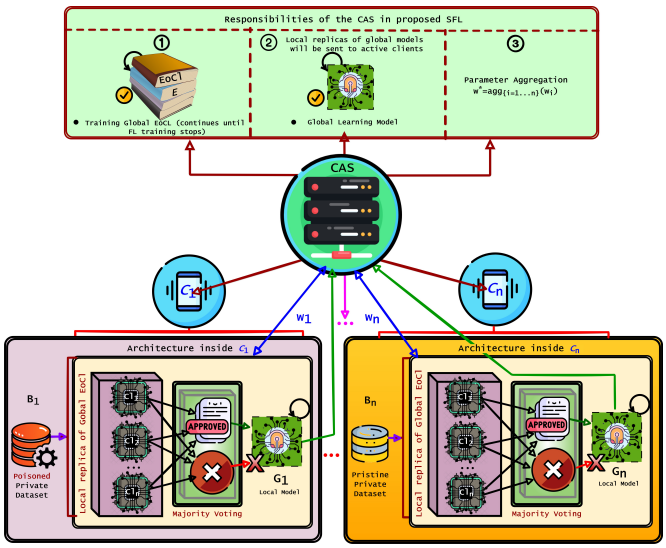



Fig. 1. Key components include (1) local replicas of the pre-trained EoCl and (2) local learning models are placed inside active client devices.

In this work, we address data poisoning from clients by using a new lightweight classifier ensemble method called Ensemble of Classifiers (EoCl) to clean their private data. The EoCl is trained globally at the Central Aggregate Server (CAS) using a variety of related datasets and continues to train until the FL process concludes. Before each global update, each active client receives the latest version of the pre-trained EoCl and a copy of the global model from the CAS. Clients then use their local EoCl to filter out any corrupted samples from their data before training their local models. We will explain these methods in detail later in the paper (see Fig. 1) and support our claims with experiments using well-known synthetic datasets.

## II. Defending Against Clients' Data Poisoning using EoCl

As already discussed the clients $\mathcal{C}_i : i = 1 \cdots n$ use their private training data $\mathcal{B}_i : i = 1 \cdots n$ to train the local models $G_{\theta_i} : i = 1 \cdots n$ (any learning models including neural networks) with respective model parameter vector $\theta_i : i = 1 \cdots n$. The distributed nature of FL raises a number of security issues in the local private data (either already stored in the client device or the client gradually gathers from external IoT devices like Glucose monitors) and learning models in the clients. For example, it is reasonable to assume that the adversaries have full control over the private data, labels, local model parameters, and model updates (broadcast to the central server over the network) for *one* or *more* clients among many legitimate FL users lying out of reach of the adversaries. Accordingly, at the local stage, the adversaries may poison the local private training data and/or change the model parameters to slowly replace the local learning models (i.e., model replacement attacks) in *one* or *more* clients over several iterations.

Data poisoning attacks in FL often take place when adversaries, which may include external actors or even legally registered malicious clients within the FL system, interfere with the training process. These attacks can take various forms, such as injecting false or misleading data points, modifying existing data points (corrupting pristine training samples $(\mathbf{x})$ and/or labels $(\mathbf{y})$), or selectively withholding certain data. Since FL relies on collaborative model training across distributed clients (each holding multi-modal private training data), attackers can exploit vulnerabilities in the training process to influence the global model's behavior [17]. For example, an attacker could deliberately inject biased data that skews the model's predictions towards their desired outcomes. Additionally, adversaries may strategically target specific clients to compromise the integrity of the global model without directly accessing the centralized server. As a result, data poisoning attacks pose significant threats to the reliability and security of FL systems, highlighting the importance of implementing robust defense mechanisms to detect and mitigate such malicious activities.

The existing defense mechanisms against poisoning attacks in FL exhibit several drawbacks. Firstly, many approaches focus on client-level defenses, such as input sanitization or outlier detection, which may not be effective against sophisticated attackers [15], [24]. Additionally, these methods often require significant computational resources, leading to high overhead and scalability issues, especially in resource-constrained client devices [7]. Furthermore, client-based defenses may not adequately address systemic vulnerabilities inherent in FL systems, such as the centralized aggregation process, which can be exploited by adversaries to influence the global model [2], [9]. Moreover, existing defenses may not adapt well to dynamic and evolving attack strategies, necessitating continual updates and enhancements [21], [24]. Overall, while current defense mechanisms provide some level of protection, they fall short in addressing the full spectrum of challenges posed by poisoning attacks in FL. Hence, they are mostly not desirable to deploy in practice, as they often either drastically harm the generalization performance, or are attack-specific, and prohibitively slow to apply [1].

*1) Proposed EoCl :* Here, we propose a robust ensemble-based input defense, known as Ensemble of Classifiers (EoCl), designed to identify corrupted private training samples utilized in training local models $G_{\theta_i}$ (mainly neural networks with $\theta_i$ as local model parameter vector) within clients $\mathcal{C}_i$. The proposed EoCl modeling involves amalgamating diverse base classifiers to produce comprehensive predictions by leveraging the unique strengths of each component. These classifiers vary in terms of training data, parameter settings, and model architecture, allowing them to capture different facets of the data or specialize in certain aspects. By combining these diverse models, EoCl creates a robust defense capable of effectively countering a range of data poisoning attacks. Notably, ensemble modeling offers flexibility, enabling the incorporation of numerous base classifiers without limitation. We integrate the EoCl as an additional component in each client, ensuring that private samples undergo verification

by the EoCl and receive a majority "*yes*" vote before being utilized in training the local model $G_{\theta_i}$ in each $\mathcal{C}_i$. However, training EoCl directly within client devices poses challenges mainly due to *two* primary reasons: (*i*) it escalates the computational load on active clients, and (*ii*) training EoCl with inadequate private training data compromises its effectiveness as an input defense mechanism.

To address this, we advocate for a global EoCl defense, rigorously trained on a variety of diverse datasets within the CAS, assuming the CAS has sufficient capacity to handle this additional activities alongside aggregation. This assumption is realistic from both execution and security perspectives, as CASs, being part of the FL service providers' infrastructure, can offer enhanced security, privacy, and robust computing facilities compared to any individual active client. Registered active clients download a local replica of this global EoCl alongside the local learning model at the start of each global iteration. By leveraging comprehensive training data, the global EoCl gains robust insights into corrupted samples, enhancing its ability to thwart malicious client insiders and adversaries from disrupting both local and global training processes. Additionally, to safeguard against adversarial activities like stealing and parameter/hyperparameter tampering of the local models $G_{\theta_i}$ we propose (*i*) locking both the local model and EoCl using a secret key provided by the CAS, and then (*ii*) housing the local model and EoCl within a dedicated CPU core (we call enclave) in each active client device.

Proposed EoCl functions as a learning model, it necessitates extensive training using diverse multi-class, multi-modal large image datasets. We call the pristine and poisoned training sets as $\mathcal{T}$ and $\mathcal{T}^\circ$ respectively. Consequently, the resulting global EoCl acquires a deep understanding of both poisoned and pristine samples. We suggest selecting diverse set of classifiers (e.g., logistic regression, support vector machines, decision tree, random forests, and multi-layer perceptrons) based on FL-system objectives, dataset characteristics, and problem nature. Details of the proposed global EoCl training strategy is discussed as below:

We assume each multi-modal training dataset has $N \in \mathbb{N}$ pristine classes denoted by $\mathcal{C}l_j : j = 1 \cdots N$. We compute hash values (using well-known hashing algorithms like MD5, SHA-256, SHA-512, etc.) of the samples from all the pristine training classes $\mathcal{C}l_j$ and apply "$\mod m$" to divide each training class (in a round-robin fashion, such that each partition contains roughly the same amount of data points) into $m$ disjoint partitions $P_{ji} : j = 1 \cdots N; i = 1 \cdots m$, where a $P_{ji}$ denotes the $i$-th partition of $\mathcal{C}l_j$. We, therefore, design the global EoCl as a set of $N$ classifier ensembles, where each ensemble contains $m$ classifier models $\{Cl_jM_i : j = 1 \cdots N; i = 1 \cdots m\}$ with diverse parameter settings, where each $Cl_jM_i$ is trained using the respective $P_{ji}$ (see Fig. 2). Here, $Cl_jM_i$ denotes the $i$-th classifier model trained on the $i$-th partition of class-$j$ or $\mathcal{C}l_j$. Next, we prepare poisoned training set $\mathcal{T}^\circ$ by poisoning approx. $20\%$ samples from each class of the pristine multi-modal training dataset $\mathcal{T}$. Next, we compute the hash values of the $\mathcal{T}^\circ$ samples and apply $\mod m$ on them to create $m$ disjoint partitions $P^\circ_{ji} : i = 1 \cdots m; j = 1 \cdots N$ in each poisoned training class $\mathcal{C}l^\circ_j : j = 1 \cdots N$. The same global EoCl model (previously used with pristine $\mathcal{T}$) with $N$ ensembles each having $m$ random forests, where each $\mathcal{C}l^\circ_j M_i$ is trained on the respective $P^\circ_{ji}$. The final prediction is determined through majority voting (Fig. 2). We then compare the accuracy between the global EoCl trained on each $\mathcal{C}l_j$ and $\mathcal{C}l^\circ_j$ to compute the accuracy metric $\delta_{\alpha_j}$ defined as below:

$$\delta_{\alpha_j} : = \alpha^\circ_j(\mathcal{C}l^\circ_j) - \alpha_j(\mathcal{C}l_j) \tag{1}$$

where $\alpha_j(\mathcal{C}l_j)$ and $\alpha^\circ_j(\mathcal{C}l^\circ_j)$ are respectively the accuracies of the global EoCl trained on each $\mathcal{C}l_j$ (from $\mathcal{T}$) and corresponding $\mathcal{C}l^\circ_j$ (from $\mathcal{T}^\circ$), shown in Fig. 2. Hence, $\delta_{\alpha_j}$ measures the accuracy variation of the global EoCl trained on each $\mathcal{C}l^\circ_j$ with regard to the same global model trained on corresponding $\mathcal{C}l_j$. A negative value of $\delta_{\alpha_j}$ indicates that the global EoCl model trained on a poisoned class experiences a decrease in accuracy. Conversely, a positive value signifies that the model trained on a poisoned dataset sees an increase in accuracy. A value of zero indicates no change in accuracy. We used diverse multi-modal datasets such as GTSRB, MNIST, CIFAR-10, and LSUN-Bedroom for training the global EoCl. Once trained, the active clients in a specific global iteration can download this pre-trained global EoCl along with a local replica of the global model. During the training of the local model $G_{\theta_i}$ in $\mathcal{C}_i$, the client's private input samples undergo scrutiny by the local pre-trained EoCl. *Only those samples garnering a majority "yes" vote from the corresponding local EoCl are permitted to participate in the local model training process* (see Fig. 1 and Fig. 2).

We recommend regularly updating the global EoCl by refining classifier selection and training it on a wider range of datasets. Additionally, local EoCls in active clients should be periodically updated after a set number of global iterations, referred to as Adaptive-EoCl (AEoCl). The timing can depend on the quality of client updates, ensuring both accuracy and security, though the final decision is up to the FL provider.

*2) Reasons for Choosing an Ensemble-based Solution:* There are broadly *five* distinct reasons behind using our proposed ensemble-based EoCl defense that are: (*a*) *Unavailability of Sufficiently Large and Diverse Training Data*: In the preceding section, we operated under the assumption that the global EoCl is trained using a diverse array of datasets. However, in practice, this assumption can sometimes prove challenging due to the limited availability of such datasets. For instance, in the context of FL-based healthcare applications focusing on extremely rare medical conditions like $47, XXX$ or Trisomy $X$—conditions characterized by the presence of additional $X$ chromosomes leading to various developmental challenges—it becomes exceedingly difficult to amass a sufficiently broad range of training data [11]. Consequently, in scenarios where training data is scarce, the hypothesis space $\mathcal{H}$ may contain numerous classifiers that exhibit identical detection accuracies when faced with specific

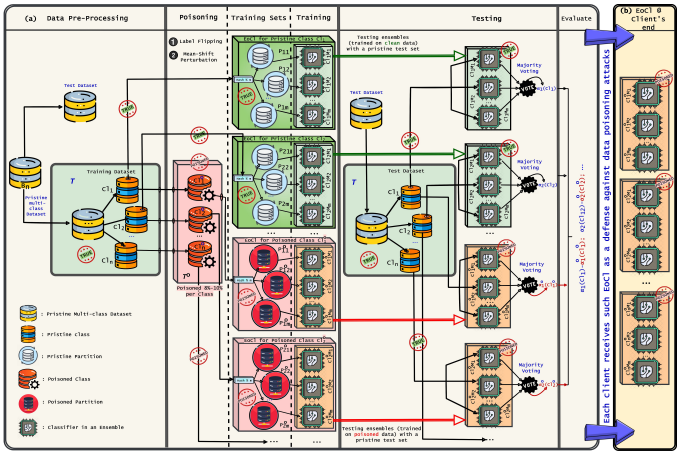



Fig. 2. **(a)** Training details of the global EoCl as a set of $N$ classifier ensembles, where each ensemble contains $m$ classifiers. **(b)** An EoCl instance received by an active client at the beginning of a global iteration. Global EoCl is trained in CAS.

*poisoning* attacks. To mitigate this risk, one approach involves constructing an ensemble comprising these accurate classifiers. By aggregating their individual predictions, the algorithm can effectively average their votes, thereby reducing the likelihood of selecting an incorrect classifier [17]. This ensemble-based strategy ultimately enhances the accuracy of *poisoning* attack detection. $(b)$ *Searching often gets Stuck to the Local Optima*: Many classification algorithms (involving neural networks like multi-layered perceptron) work by performing some kinds of local search that might get stuck to the local optima. For example, decision tree classifiers use a greedy splitting rule to grow the decision tree. Now, in many cases, it has been noticed that despite having sufficient training data the respective classification algorithm may find it computationally hard to get the best hypothesis. An ensemble involving multiple such classifiers, on the other hand, can simultaneously initiate the search from different starting points, which may provide a better approximation to the true unknown function. $(c)$ *Enhancement of Reliability*: Ensemble models address common issues like variance, bias, and noise encountered in single base models. Bias arises from oversimplified assumptions in the learning algorithm, leading to underfitting by missing relevant feature-target relations. Conversely, variance stems from excessive model complexity, causing overfitting as the model learns noise in the training data. Ensemble methods mitigate biases and reduce variance by combining predictions from multiple models, giving weight to a wider range of features. This results in a more reliable and robust model with improved generalization capabilities. Here, robustness is characterized by low variance, signifying the resilience of the EoCl against minor inconsistencies in the training data. Even if overall accuracy remains unchanged, the EoCl effectively reduces variance or prediction spread, contributing to its stability and consistency. $(d)$ Finally, it is often hard to find any specific hypothesis $\mathcal{H}$ that accurately matches with the true unknown function. An ensemble, on the other hand, may use some weighted sums of the hypotheses from $\mathcal{H}$ to achieve a better approximation of the true unknown function. By harnessing the predictive capabilities of ensemble members, the proposed EoCl demonstrates enhanced detection capabilities across a broad spectrum of data poisoning attacks, ensuring heightened generalizability and accuracy.

Apart from that, ensemble-based methods significantly enhance model performance by combining existing models, crucial for domains such as healthcare, finance, and autonomous vehicles. They are versatile, handling both classification and regression tasks while integrating diverse model types, allowing seamless adaptation to evolving ML landscapes. Moreover, ensembles excel in managing large datasets, providing robustness against challenges like high dimensionality and missing values. Their parallelizable nature enables efficient training across multiple processors or distributed machines, maximizing resource utilization and aligning with decentralized computational trends, making them ideal for complex computations benefiting from collective intelligence.

The active local clients receive the local replicas of the pre-trained global EoCl along with the local versions $G_{\theta_i}$ of the global learning model at the beginning of any global iteration. Fig. 2 presents an overview of the EoCl training. Benefits of our ensemble-based defense EoCl includes: (i) EoCl does not hamper the natural execution of the local models and does not put any extra computational burden on individual client; and (ii) EoCl can handle the untargeted poisoning attacks (along with targeted attacks), where an attacker with limited knowledge and resources randomly alters the data to reduce the accuracy of the resulting model. We test the effectiveness of EoCl using the following threat model discussed in subsection II-3.

*3) Threat Model for EoCl :* In this paper, we focus on online untargeted black-box data poisoning attack as it is the most common and relevant to production-phase FL deployments and can remain undetected for a longer duration of time among a larger population of FL clients. Specifically, the attacker is interested in generic misclassification (untargeted) rather than specific misclassification (targeted). For the untargeted attacks, we assume the attacker needs to adjust with his/her insufficient knowledge of $\mathcal{C}_i$ and the corresponding $\mathcal{G}_{\theta_i}$. An attacker launches specific poisoning attacks on the private data of $\mathcal{C}_i$ either by altering information (even at the feature level through data poisoning) and/or label (through label flipping) in approximately $5\% - 20\%$ random samples of $\mathcal{B}_i$. Next, the pre-trained EoCl performs "majority voting" of the incoming samples to decide about its legitimacy.

## III. Experiments and Implementation

In this work, we aim to improve the usability of mobile devices by advancing image classification tasks. We use the following datasets to test our proposed strategy.

### A. Dataset Description

$(a)$ *GTSRB* [20]: This dataset has 43 RGB traffic sign image classes at $32 \times 32$ pixel resolution, with $34,799$ training, $4,410$

validation, and $12,630$ test images. It is highly imbalanced, making classification challenging. (*b*) *MNIST* [5]: A benchmark of $70,000$ grayscale $28\times28$ images of handwritten digits $(0-9)$, with $60,000$ for training and $10,000$ for testing. (*c*) *CIFAR-10* [10]: This dataset includes $60,000$ $32\times32$ color images in 10 classes (airplanes, cars, etc.), with $50,000$ for training and $10,000$ for testing. (*d*) *LSUN-Bedroom* [25]: Over 3 million high-resolution bedroom images, primarily used in image generation and scene understanding tasks.

### B. Training Global EoCl

We used standard Python library of TensorFlow for the present implementation. As already discussed in Section II, the objective of the ensemble-based EoCl defense is to combine the decisions from multiple classifiers to improve accuracy and resilience of the overall FL training. In this work, we primarily used homogeneous classifier ensembles, either with Random Forests (RFs) or SVMs, each with different parameter settings for simplicity. However, using ensembles of heterogeneous classifiers is also recommended for cases where it may be more suitable (will be reported in the future publications).

As already discussed, we have used many multi-modal datasets including GTSRB, MNIST, CIFFAR-10, and LSUN-Bedroom for validating our claims. Our experimental settings include: (*i*) we design EoCl with $N$ ensembles ($N$ be the number of distinct classes or $\mathcal{C}l_j: j = 1\cdots N$ in the used datasets), where each ensemble has $m$ classifiers (discussed in Section Fig. 2). We denote it as $\mathcal{C}l_jM_i: j = 1\cdots N; i = 1\cdots m$, which means $i$-th classifier $M_i$ of ensemble-$j$ for class-$j$ $(\mathcal{C}l_j)$. Here, $M_i$ can either be a RF or a SVM. For example, in case of RF $\mathcal{C}l_jM_i$ becomes $\mathcal{C}l_jRF_i$. Here, $m$ value has intentionally been kept small since the classes in the used datasets (Section III-A) do not contain adequate training examples. *(ii)* We used a variety of data poisoning methods such as mean-shift perturbations [17], [22] and label flipping [16]. *(iii)* We vary the percentage of poisoned samples (using mean-shift perturbations and label flipping) $\varepsilon$ between $8\%$ and $20\%$ (see Fig. 2) in the pristine classes to prepare poisoned training classes. (*iv*) The hash functions used for data point assignments are extremely fast and lightweight. (*v*) All $m$ classifiers in each ensemble are trained independently on disjoint partitions. Such training can be performed parallely. Furthermore, we tested our ensemble approach under a worst-case scenario by using the outdated MD5 hash function to assign data points to disjoint partitions $(P_{ij})$ in different classes. This was done to see if our method could still defend against poisoning attacks. Lastly, we set up each classifier according to the best practices outlined in the current literature.

Detailed EoCl training is explained in Section II-1 and Fig. 2. For the GTSRB dataset (with 43 unique classes, $\mathcal{C}l_j: j = 1\cdots43$), our EoCl contains 43 ensembles ($N = 43$), where each ensemble has $m \in \{3, 5\}$ classifiers. Similarly, for MNIST and CIFAR-10 datasets, we set $N = 10$ (since each dataset has 10 distinct classes) with $m \in \{3, 5\}$ due to insufficient training samples in individual classes. For the LSUN-Bedroom dataset, we use $m \in \{3, 5, 7, 9, 11, 17, 21\}$ as each class contains adequate training samples. Next, to prepare the training set for each of the ensembles assigned to $\mathcal{C}l_j: j = 1\cdots N$, we first compute the MD5 hashes of all the training samples in each $\mathcal{C}l_j$. We then convert each hash value into an integer and apply $\mod m$ on each hash value of $\mathcal{C}l_i$, dividing $\mathcal{C}l_i$ into $m$ disjoint partitions $P_{ji}$, where $j = 1\cdots N$ and $i = 1\cdots m$. Each such pristine partition is then used to train the corresponding $\mathcal{C}l_jM_i$, for example, partition $P_{13}$ is used to train the corresponding $\mathcal{C}l_1M_3$.

Subsequently, we will prepare corrupted training samples for each class within the multi-modal dataset. This process involves employing mean-shift perturbation [17], [22] and label flipping techniques [6], [14]. We systematically incorporate between $8\%$ and $20\%$ corrupted samples in each class, thereby creating the corrupted classes denoted as $\mathcal{C}l^{\circ}_j: j = 1\cdots N$. We partitioned the corrupted classes $P^{\circ}_{ji}: i = 1\ldots m$ using the same method applied to the pristine classes $\mathcal{C}l_j: j = 1\cdots m$, as described before (see Fig. 2). We rigorously train all classifier ensembles in the current EoCl implementation using both pristine and corrupted classes from the baseline datasets (in Section III-A) in the CAS. The accuracies are computed as described in Section II and the results are summarized in Table I. Such global EoCl training will continue as long as the training of the overall FL system continues and the active clients' current EoCl models are periodically replaced by the capable intermediate EoCl (mainly same ensemble of classifiers with different parameter settings) to defeat the possibility of any guessing attacks on the components of the EoCl, although such attacks are highly unrealistic and very hard to perform. We prepare dataset for training the global EoCl by mixing pristine and poisoned (varying $\epsilon$ between $8\%$ and $20\%$ per class) samples.

Table I presents the accuracies (averaged over 5000 iterations) of various filter ensembles, including RFs and SVMs, against corrupted training datasets (MNIST, GTSRB, and LSUN-Bedroom) with 5%–20% poisoned samples generated by mean-shift perturbations. The columns for $m = 1$ (monolithic models) at $\mathcal{E} = 0$ (no poisoning, highlighted gray) show accuracies against these perturbations. Each cell is split into two parts: the top part indicates the difference $\delta_\alpha$ between accuracies of the EoCl trained on pristine versus poisoned datasets, while the bottom part reports the accuracy $\alpha^{\circ}$ $(\mathcal{C}l^{\circ})$ (averaged over 5000 iterations) for ensembles trained on poisoned sets with $\mathcal{E}$ ranging from 5% to 20%.

Our results show significant variation in accuracy across the four baseline datasets under mean-shift perturbations [17], though only *three* are presented in Table I due to space limits. Monolithic models (with $m = 1$) using RF achieved 96.091% accuracy for MNIST, 97.312% for GTSRB, and 98.347% for LSUN-Bedroom. These differences reflect dataset diversity and were consistent across configurations, with performance peaking at $\mathcal{E} = 20\%$. Additionally, we identified *two* key trends from these results.

$Trend\#1$: As the proportion of poisoned samples $(\mathcal{E})$ increases, accuracy $(\delta_\alpha)$ decreases consistently for both RF and SVM ensembles. In the smallest RF ensemble ($m = 3$),

$\delta_\alpha$ drops from $-1.12$ to $-15.01$ for MNIST, $-1.20$ to $-14.31$ for GTSRB, and $-2.69$ to $-19.18$ for LSUN-Bedroom as $\mathcal{E}$ rises from 5% to 20%. SVM ensembles show similar trends across all datasets (see Table I).

*Trend#2:* Increasing the number of classifiers $m$ in the ensemble reduces the accuracy drop. For instance, with $\mathcal{E} = 20\%$, the accuracy improves by approximately 60% for MNIST ($-23.09$ to $-4.09$), 58% for GTSRB ($-26.31$ to $-9.56$), and 65% for LSUN-Bedroom ($-26.09$ to $-6.15$) as the number of RF classifiers increases from 1 to 5 or more. This trend is also observed in SVM ensembles with baseline datasets (see Table I).

The entries highlighted in blue in Table I represent the accuracy values of the most effective filter ensembles when tested against poisoned baseline datasets, with $\mathcal{E}$ ranging from 5% to 20%. For instance, the ensemble consisting of 3 Random Forests (RFs) ($m = 3$) achieved the highest performance, demonstrating an accuracy of 94.89% when evaluated on a baseline MNIST dataset with 5% poisoned samples per class. This indicates that the ensemble can accurately detect a corrupted sample with 94.89% precision. Similar interpretations apply to the other highlighted values in Table I. In the next step, during each global iteration, each active client $\mathcal{C}_i$ receives a pre-trained global `EoCl` one at a time. This `EoCl` in an active $\mathcal{C}_i$ acts as a local defense mechanism. The client's private training data must pass through this local `EoCl` before being used to participate in the training of local model $G_{\theta_i}$. This process helps safeguard the global FL training by filtering out the local poisoned samples.

### C. Accuracy of the Local Learning Models

In the present work, we conducted the evaluation of our `EoCl` using the following deep CNN-based classification models as global model (between Eq. 2 and Eq. 6), taking one at a time. The `EoCl` is trained sequentially on the baseline datasets, ensuring that each dataset is addressed individually, one at a time.

$$\texttt{Conv(128,6,6)} + \texttt{ReLU} \tag{2}$$
$$\texttt{Conv(128,5,5)} + \texttt{ReLU} \tag{3}$$
$$\texttt{Conv(64,5,5)} + \texttt{ReLU} \tag{4}$$
$$\texttt{FC(300)} + \texttt{ReLU} \tag{5}$$
$$\texttt{FC(128)} + \texttt{ReLU} \tag{6}$$

In this work, we consider our experimental SFL framework has 50 clients ($\mathcal{C}_i : i = 1 \cdots 50$) connected to the CAS, that runs global models described between Eq. 2 and Eq. 6 individually, one at a time. Subsequently, each of the active clients runs the local version of the global model with different parameter settings (such as different learning rates, weights, and biases). For example, for the global model in Eq. 2 each of the clients runs local model $G_{\theta_i}$ as `Conv(128,6,6)+ReLU` with different learning rate ($\eta$), weight ($\omega$) and bias ($\beta$) values. Such changes in parameter settings are mainly because of the difference in the private training data set stored in different clients. We also assume each active client $\mathcal{C}_i$ performs $\tau \in \mathbb{N}$ local iterations. The training datasets are evenly distributed among 50 clients at any time. In the IID setting [18], [19], each client receives a randomly assigned, uniformly distributed portion of data across all the classes in each dataset. For the non-IID setting [18], [19], the data is sorted by class and divided to create two extreme scenarios: (*i*) 1-class non-IID, where each client receives data from only a single class, and (*ii*) 2-class non-IID, where the sorted data is divided into 50 partitions, with each client randomly receiving five partitions from the two classes (similar settings used in [26]).

CAS computes the average of the parameter updates sent by $n = 50$ different active clients, participating in a global iteration (equivalent to FedAvg. [12], see Fig. 1 and Fig. 2). Let $\gamma$ be the batch size and $\tau$ be the number of local iterations. We used the following parameter settings in the clients: (*i*) for MNIST data $\gamma = 10$ or 100, $\tau = 1$ or 5, $\eta = 0.01$ with a decay rate of 0.995; (*ii*) for CIFAR-10, $\gamma = 10$ or 100, $\tau = 1$ or 5, $\eta = 0.1$ with a decay rate of 0.992; (*iii*) for GTSRB, $\gamma = 10$ or 100, $\tau = 1$ or 5, $\eta = 0.1$ with a decay rate 0.994; and (*iv*) for LSUN-Bedroom, $\gamma = 10$ or 100, $\tau = 1$ or $5, \eta = 0.01$ with a decay rate 0.99. For a client $\mathcal{C}_i$ let $U$ be the number of global iterations.

For all the five base models (discussed between Eq. 2 and Eq. 6) trained using pristine datasets, the computations converge to a higher level of test-set accuracy given in Table II. For example, for the CNN models `FC(128)+ReLU` trained using pristine MNIST (under IID assumption) with $\tau = 5$ and $\gamma = 100$ eventually reaches 99.13% accuracy (approx.) after 3500 global iterations, highlighted blue (and had not improved further after 4000 global rounds $U$). However, the same `FC(128)+ReLU` model with $\tau = 5$ and $\gamma = 100$ reaches an accuracy of 99.29% after the same 3500 global rounds. Similarly, `Conv(64,5,5)+ReLU` with $\tau = 5$ and $\gamma = 100$ (highlighted blue in the pristine CIFAR-10 part in Table II) converges to 99.19% test accuracy after $U = 4100$ global iterations for IID pristine CIFAR-10 among 50 clients. Other entries in Table II can be interpreted similarly.

We randomly corrupt 8% to 20% of samples in each clean training class across all baseline datasets (one at a time, as detailed in Section III-A). Our `EoCl` model is trained using one corrupted dataset at a time. We then place the trained `EoCl` between the private data and the local model on each active client and perform testing. The results, shown in Table III, indicate that the `Conv(128,5,5)+ReLU` model performs best in the non-IID setting with $\tau = 5$, $\gamma = 1$, and $\mathcal{E} = 5\%$ (highlighted blue in the LSUN-Bedroom section of Table III). Specifically, `Conv(128,5,5)+ReLU` achieves approximately 95.59% accuracy (highlighted blue in Table I) after 5329 global iterations (highlighted red in Table III) when the corrupted LSUN-Bedroom data (with $\mathcal{E} = 5\%$) is filtered using a local pre-trained ensemble of 5 random forests (RFs) per client. Similarly, with $\mathcal{E} = 20\%$ and a 2-class setting, the same model achieves 87.96% accuracy (highlighted blue in Table I) after 5329 global iterations when using an ensemble of 11 RFs per client for filtering (see

TABLE I

ACCURACY OF DIFFERENT FILTER ENSEMBLES (USING A VARYING NUMBER OF CLASSIFIERS OR $m$) AGAINST VARIOUS CORRUPTED TRAINING DATA (INCLUDING MNIST, GTSRB, AND LSUN-BEDROOM) CONTAINING $5\% - 20\%$ POISONED SAMPLES (PREPARED USING MEAN-SHIFT PERTURBATIONS [17]) PER CLASS, I.E., $\mathcal{E} \in (0, 0.2)$. THIS COMPUTATION IS AVERAGED OVER 50 ACTIVE CLIENTS AFTER 1000 GLOBAL ITERATIONS.

| MNIST | | | | | | | GTSRB | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\mathcal{E}\%$ | ⟵ No. of RFs ⟶ | | | ⟵ No. of SVMs ⟶ | | | $\mathcal{E}\%$ | ⟵ No. of RFs ⟶ | | | ⟵ No. of SVMs ⟶ | | |
| ↓ | $m=1$ | $m=3$ | $m=5$ | $m=1$ | $m=3$ | $m=5$ | ↓ | $m=1$ | $m=3$ | $m=5$ | $m=1$ | $m=3$ | $m=5$ |
| 0 | 0.00<br>96.09 | 0.00<br>96.01 | 0.00<br>93.33 | 0.00<br>95.71 | 0.00<br>93.01 | 0.00<br>92.12 | 0 | 0.00<br>97.31 | 0.00<br>95.11 | 0.00<br>94.01 | 0.00<br>96.08 | 0.00<br>95.21 | 0.00<br>94.00 |
| 5 | −3.71<br>92.37 | −1.12<br>**94.89** | −0.23<br>93.09 | −3.92<br>91.79 | −1.77<br>91.23 | −0.30<br>91.82 | 5 | −3.09<br>**94.22** | −1.20<br>93.91 | −0.40<br>93.61 | −3.09<br>92.99 | −1.00<br>94.21 | −0.89<br>93.11 |
| 10 | −2.91<br>93.17 | −1.31<br>**94.69** | −0.91<br>92.42 | −3.08<br>92.62 | −1.90<br>91.10 | −0.92<br>91.20 | 10 | −7.21<br>90.10 | −5.33<br>89.78 | −1.61<br>**92.40** | −7.99<br>88.09 | −5.58<br>89.63 | −1.65<br>92.34 |
| 15 | −10.71<br>85.38 | −6.01<br>90.00 | −3.08<br>**90.24** | −11.98<br>83.73 | −7.79<br>85.22 | −5.10<br>87.02 | 15 | −10.18<br>87.14 | −9.39<br>85.72 | −5.87<br>88.14 | −10.91<br>85.17 | −9.22<br>85.99 | −4.91<br>**89.091** |
| 20 | −23.09<br>73.00 | −15.01<br>81.00 | −4.09<br>**89.24** | −24.98<br>70.73 | −16.01<br>77.00 | −6.10<br>86.02 | 20 | −26.31<br>71.01 | −14.31<br>80.80 | −9.56<br>84.45 | −25.11<br>70.97 | −18.89<br>76.32 | −8.81<br>**85.19** |

| LSUN-Bedroom | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\mathcal{E}\%$ | ⟵ No. of RFs ⟶ | | | | | | $\mathcal{E}\%$ | ⟵ No. of SVMs ⟶ | | | | | |
| ↓ | $m=1$ | $m=3$ | $m=5$ | $m=9$ | $m=11$ | $m=21$ | ↓ | $m=1$ | $m=3$ | $m=5$ | $m=9$ | $m=11$ | $m=21$ |
| 0 | 0.00<br>98.34 | 0.00<br>97.13 | 0.00<br>96.18 | 0.00<br>96.14 | 0.00<br>94.11 | 0.00<br>93.16 | 0 | 0.00<br>98.21 | 0.00<br>97.39 | 0.00<br>97.11 | 0.00<br>95.48 | 0.00<br>95.67 | 0.00<br>94.11 |
| 5 | −3.47<br>94.87 | −2.69<br>95.44 | −2.58<br>**95.59** | −1.41<br>94.72 | −1.88<br>92.23 | −0.81<br>92.35 | 5 | −3.89<br>**94.32** | −3.48<br>93.91 | −2.85<br>94.25 | −2.71<br>92.71 | −1.91<br>93.76 | −0.88<br>93.23 |
| 10 | −7.99<br>90.35 | −6.68<br>**92.45** | −6.49<br>89.68 | −4.67<br>91.46 | −1.99<br>92.12 | −1.82<br>91.39 | 10 | −8.76<br>89.45 | −8.21<br>89.18 | −7.39<br>89.72 | −5.86<br>89.56 | −5.13<br>**90.54** | −5.02<br>89.09 |
| 15 | −14.48<br>83.86 | −9.57<br>87.56 | −7.87<br>88.31 | −7.23<br>88.91 | −4.01<br>**90.10** | −3.09<br>90.07 | 15 | −18.36<br>79.84 | −16.09<br>81.30 | −12.23<br>**84.884** | −12.09<br>83.334 | −11.11<br>84.559 | −11.99<br>82.119 |
| 20 | −26.091<br>72.256 | −19.183<br>77.951 | −15.177<br>81.004 | −9.079<br>87.062 | −6.157<br>**87.961** | −6.153<br>87.012 | 20 | −27.511<br>**70.700** | −26.99<br>70.40 | −27.09<br>70.02 | −27.33<br>68.09 | −27.11<br>68.55 | −26.33<br>67.78 |

Table III). These results strongly support the effectiveness and accuracy of our proposed `EoCl` strategy.

## IV. CONCLUSION

In conclusion, this work presents a novel approach to enhancing security in FL by addressing the critical issue of data poisoning. By introducing the `EoCl` as a lightweight, global defense mechanism, we aim to protect both the integrity of clients' private data and the CAS. Our approach ensures that data poisoning attempts are mitigated through rigorous cleansing of clients' data, leveraging the global `EoCl` that is continually refined and updated to enhance its effectiveness.

We propose an adaptive strategy with regular global `EoCl` updates and periodic local classifier refreshes, based on parameter quality and loss metrics. This approach enhances FL system robustness against malicious data. Our experiments on public datasets validate its effectiveness, laying the groundwork for future improvements. In upcoming work, we will address client-side model poisoning to further improve FL accuracy.

## REFERENCES


[1] Mohammed H Alsharif, Raju Kannadasan, Wei Wei, Kottakkaran Sooppy Nisar, and Abdel-Haleem Abdel-Aty. A contemporary survey of recent advances in federated learning: Taxonomies, applications, and challenges. *Internet of Things*, page 101251, 2024.

[2] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International conference on artificial intelligence and statistics*, pages 2938–2948. PMLR, 2020.

[3] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643. PMLR, 2019.

[4] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[5] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and Andre Van Schaik. Emnist: Extending mnist to handwritten letters. In *2017 international joint conference on neural networks (IJCNN)*, pages 2921–2926. IEEE, 2017.

[6] Ernesto Damiani and Claudio A Ardagna. Certified machine-learning models. In *International Conference on Current Trends in Theory and Practice of Informatics*, pages 3–15. Springer, 2020.

[7] Dongcai Du, Wei Zhao, Linna Wei, Siyang Lu, and Xuangou Wu. A lightweight homomorphic encryption federated learning based on blockchain in iov. In *2022 IEEE Smartworld, Ubiquitous Intelligence & Computing, Scalable Computing & Communications, Digital Twin, Privacy Computing, Metaverse, Autonomous & Trusted Vehicles (SmartWorld/UIC/ScalCom/DigitalTwin/PriComp/Meta)*, pages 1001–1007. IEEE, 2022.

[8] Briland Hitaj, Giuseppe Ateniese, and Fernando Perez-Cruz. Deep models under the gan: information leakage from collaborative deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pages 603–618, 2017.

[9] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *Foundations and trends® in machine learning*, 14(1–2):1–210, 2021.

[10] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[11] M MacDonald, T Hassold, J Harvey, LH Wang, NE Morton, and P Jacobs. The origin of 47, xxy and 47, xxx aneuploidy: heterogeneous mechanisms and role of aberrant recombination. *Human molecular genetics*, 3(8):1365–1371, 1994.

[12] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[13] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 691–706. IEEE, 2019.

[14] Mehran Mozaffari-Kermani, Susmita Sur-Kolay, Anand Raghunathan, and Niraj K Jha. Systematic poisoning attacks on and defenses for machine learning in healthcare. *IEEE journal of biomedical and health informatics*, 19(6):1893–1905, 2014.

[15] Amirhossein Reisizadeh, Farzan Farnia, Ramtin Pedarsani, and Ali Jadbabaie. Robust federated learning: The case of affine distribution shifts. *Advances in Neural Information Processing Systems*, 33:21554–21565, 2020.

[16] Elan Rosenfeld, Ezra Winston, Pradeep Ravikumar, and Zico Kolter. Certified robustness to label-flipping attacks via randomized smoothing.

TABLE II

ACCURACY OF THE MODELS ON THE PRISTINE BASELINE DATASETS BOTH WITH IID AND NON-IID ASSUMPTIONS. BLUE AND RED HIGHLIGHTS ARE RESPECTIVELY THE VALUES CORRESPONDING TO THE BEST MODELS FOR IID AND NON-IID ASSUMPTIONS.

| Pristine MNIST ⟶ (Accuracy 99%, approx.) | | | | | | Pristine CIFAR-10 ⟶ (Accuracy 99%, approx.) | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Models | $\tau$ | $\gamma$ | $U$ | IID | non-IID (1-Class) | Models | $\tau$ | $\gamma$ | $U$ | IID | non-IID (2-Class) |
| Conv(128,6,6)+ ReLU | 1 | 10 | 4591 | 7921 | 2412 | Conv(128,6,6)+ ReLU | 1 | 10 | 5300 | 8194 | 9312 |
| | 5 | 100 | 4510 | 3691 | 1344 | | 5 | 100 | 5100 | 5311 | 9000 |
| Conv(128,5,5) + ReLU | 1 | 10 | 4315 | 2179 | 2123 | Conv(128,5,5) + ReLU | 1 | 10 | 5120 | 5019 | 9213 |
| | 5 | 100 | 5430 | 4973 | 1145 | | 5 | 100 | 5000 | 4316 | 7421 |
| Conv(64,5,5) + ReLU | 1 | 10 | 5023 | 1175 | 3212 | Conv(64,5,5) + ReLU | 1 | 10 | 5000 | 1290 | 5481 |
| | 5 | 100 | 4560 | 955 | 3241 | | **5** | **100** | **4100** | **45** | 1192 |
| FC(300) + ReLU | 1 | 10 | 4150 | 812 | 1134 | FC(300) + ReLU | 1 | 10 | 5050 | 1171 | 5721 |
| | 5 | 100 | 4280 | 400 | 1234 | | 5 | 100 | 5280 | 1311 | 2845 |
| FC(128) + ReLU | 1 | 10 | 4200 | 193 | 1322 | FC(128) + ReLU | 1 | 10 | 5200 | 170 | 3780 |
| | **5** | **100** | **3500** | **39** | **187** | | **5** | **100** | **5500** | 117 | **182** |
| **Pristine LSUN-Bedroom ⟶ (Accuracy 96%, approx.)** | | | | | | **Pristine GTSRB ⟶ (Accuracy 99%, approx.)** | | | | | |
| Conv(128,6,6) + ReLU | 1 | 10 | 6900 | 5096 | 8912 | Conv(128,6,6) + ReLU | 1 | 10 | 5141 | 4510 | 7832 |
| | 5 | 100 | 7810 | 4911 | 8192 | | **5** | **100** | **5310** | **121** | 1129 |
| Conv(128,5,5) + ReLU | 1 | 10 | 5715 | 3719 | 8721 | Conv(128,5,5) + ReLU | 1 | 10 | 5315 | 3519 | 3910 |
| | 5 | 100 | 7320 | 5213 | 7992 | | **5** | **100** | **5130** | 3951 | **192** |
| Conv(64,5,5) + ReLU | 1 | 10 | 5912 | 1348 | 8361 | Conv(64,5,5) + ReLU | 1 | 10 | 5150 | 4481 | 8723 |
| | **5** | **100** | **6123** | 970 | **170** | | 5 | 100 | 5160 | 3691 | 7823 |
| FC(300) + ReLU | 1 | 10 | 6150 | 817 | 4821 | FC(300) + ReLU | 1 | 10 | 5150 | 4126 | 5718 |
| | 5 | 100 | 6280 | 311 | 1113 | | 5 | 100 | 5280 | 1143 | 1982 |
| FC(128) + ReLU | 1 | 10 | 1200 | 252 | 2391 | FC(128) + ReLU | 1 | 10 | 5200 | 1158 | 2659 |
| | **5** | **100** | **3500** | **87** | 1384 | | 5 | 100 | 5500 | 837 | 3281 |

TABLE III

ACCURACY OF THE MODELS ON THE CORRUPTED BASELINE DATASETS FILTERED BY THE EOCL. THE BEST VALUES ARE HIGHLIGHTED IN BLUE AND RED FOR IID AND NON-IID SETTINGS RESPECTIVELY. WE SENT THE CORRUPTED DATASETS THROUGH THE BEST FILTER ENSEMBLES (SEE TABLE I) BEFORE TRAINING THE LOCAL MODELS.

| Corrupted MNIST: (${}^1\varepsilon = 5\%$; ${}^2\varepsilon = 20\%$) | | | | | | | | Corrupted CIFAR-10: (${}^1\varepsilon = 5\%$; ${}^2\varepsilon = 20\%$) | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Models | $\tau$ | $\gamma$ | $U$ | IID$^1$ | IID$^2$ | non-IID$^1$ 1-C | non-IID$^2$ 2-C | Models | $\tau$ | $\gamma$ | $U$ | IID$^1$ | IID$^2$ | non-IID$^1$ 1-C | non-IID$^2$ 2-C |
| Conv(128,6,6) + ReLU | 1 | 10 | 5100 | 3318 | 3689 | 3901 | 3923 | Conv(128,6,6) + ReLU | 1 | 10 | 5100 | 3781 | 4129 | 4721 | 3891 |
| | 5 | 100 | 5100 | **213** | **232** | 2109 | 1123 | | 5 | 100 | 5100 | **189** | **202** | 3019 | 2910 |
| Conv(128,5,5) + ReLU | 1 | 10 | 5319 | 3100 | 2309 | 3011 | 1102 | Conv(128,5,5) + ReLU | 1 | 10 | 5100 | 1370 | 2301 | 2790 | 2890 |
| | 5 | 100 | 5318 | 301 | 311 | **301** | **201** | | 5 | 100 | 5210 | 1178 | 1980 | 189 | **211** |
| Conv(64,5,5) + ReLU | 1 | 10 | 5411 | 1293 | 1530 | 1689 | 1487 | Conv(64,5,5) + ReLU | 1 | 10 | 5100 | 1197 | 1109 | **289** | **303** |
| | 5 | 100 | 5100 | 2135 | 1923 | 1389 | 1823 | | 5 | 100 | 5100 | 1570 | 1678 | 2019 | 2908 |
| FC(300) + ReLU | 1 | 10 | 5100 | 1113 | 1511 | 1012 | 1421 | FC(300) + ReLU | 1 | 10 | 5100 | 1170 | 1790 | 1890 | 1987 |
| | 5 | 100 | 5341 | 1323 | 1192 | 1295 | 1421 | | 5 | 100 | 5100 | 2029 | 2870 | 2091 | 2910 |
| FC(128) + ReLU | 1 | 10 | 5200 | 1340 | 1522 | 1362 | 1291 | FC(128) + ReLU | 1 | 10 | 5200 | 3019 | 2901 | 2819 | 2901 |
| | 5 | 100 | 5100 | 1243 | 1523 | 1986 | 1782 | | 5 | 100 | 5100 | 1589 | 1530 | 1783 | 1879 |
| **Corrupted LSUN-Bedroom: (${}^1\varepsilon = 5\%$; ${}^2\varepsilon = 20\%$)** | | | | | | | | **Corrupted GTSRB: (${}^1\varepsilon = 5\%$; ${}^2\varepsilon = 20\%$)** | | | | | | | |
| Conv(128,6,6) + ReLU | 1 | 10 | 5310 | 3321 | 3461 | 3591 | 4319 | Conv(128,6,6) + ReLU | 1 | 10 | 5400 | 2390 | 1190 | 2390 | 3210 |
| | 5 | 100 | 5310 | 1456 | 1563 | 1567 | 2346 | | 5 | 100 | 5400 | 1297 | 1730 | 2390 | 3100 |
| Conv(128,5,5) + ReLU | 1 | 10 | 5329 | 1213 | 1256 | 1678 | 1823 | Conv(128,5,5) + ReLU | 1 | 10 | 5400 | **147** | **129** | 1022 | 1190 |
| | 5 | 100 | 5329 | 1119 | 1245 | **319** | **336** | | 5 | 100 | 5500 | 1100 | 1290 | 2190 | 3210 |
| Conv(64,5,5) + ReLU | 1 | 10 | 5390 | 1563 | 1563 | 1980 | 1855 | Conv(64,5,5) + ReLU | 1 | 10 | 5290 | 1300 | 1289 | 2300 | 2890 |
| | 5 | 100 | 5300 | 1231 | 1267 | 1678 | 1980 | | 5 | 100 | 5300 | 1129 | 1236 | **276** | **217** |
| FC(300) + ReLU | 1 | 10 | 5300 | 1123 | 1562 | 1789 | 2190 | FC(300) + ReLU | 1 | 10 | 5400 | 1113 | 1522 | 2358 | 2890 |
| | 5 | 100 | 5300 | 1278 | 1567 | 189 | 2198 | | 5 | 100 | 5400 | 1189 | 1693 | 2879 | 2945 |
| FC(128) + ReLU | 1 | 10 | 5300 | 1783 | 1564 | 1987 | 2357 | FC(128) + ReLU | 1 | 10 | 5400 | 1136 | 1000 | 2831 | 2000 |
| | 5 | 100 | 5400 | **318** | **202** | 1109 | 1000 | | 5 | 100 | 5400 | 1200 | 1000 | 2378 | 2980 |


In *International Conference on Machine Learning*, pages 8230–8241. PMLR, 2020.

[17] Arunava Roy and Dipankar Dasgupta. Defending learning systems against mean-shift perturbations. *IEEE Transactions on Artificial Intelligence*, 2024.

[18] Arunava Roy and Dipankar Dasgupta. A distributed conditional wasserstein deep convolutional relativistic loss generative adversarial network with improved convergence. *IEEE Transactions on Artificial Intelligence*, 2024.

[19] Arunava Roy and Dipankar Dasgupta. Drd-gan: A novel distributed conditional wasserstein deep convolutional relativistic discriminator gan with improved convergence. *ACM Transactions on Probabilistic Machine Learning*, 2024.

[20] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: a multi-class classification competition. In *The 2011 international joint conference on neural networks*, pages 1453–1460. IEEE, 2011.

[21] Jacob Steinhardt, Pang Wei W Koh, and Percy S Liang. Certified defenses for data poisoning attacks. *Advances in neural information processing systems*, 30, 2017.

[22] Florian Tramèr, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. The space of transferable adversarial examples. *arXiv preprint arXiv:1704.03453*, 2017.

[23] Stacey Truex, Nathalie Baracaldo, Ali Anwar, Thomas Steinke, Heiko Ludwig, Rui Zhang, and Yi Zhou. A hybrid approach to privacy-preserving federated learning. In *Proceedings of the 12th ACM workshop on artificial intelligence and security*, pages 1–11, 2019.

[24] Yichen Wan, Youyang Qu, Wei Ni, Yong Xiang, Longxiang Gao, and Ekram Hossain. Data and model poisoning backdoor attacks on wireless federated learning, and the defense mechanisms: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 2024.

[25] Fisher Yu, Yinda Zhang, Shuran Song, Ari Seff, and Jianxiong Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop. *CoRR*, abs/1506.03365, 2015.

[26] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582*, 2018.